Cayleb Alvarez-James

AI Agent Systems Engineer — Autonomous Agents, Agent Harnesses & Self-Improving Infrastructure

Middletown, Connecticut, USA | caylebalvarezjames@gmail.com

github.com/cayleb-james2008 | linkedin.com/in/cayleb-alvarez-james-99850531b | cayleb-james2008.github.io

Summary

AI agent systems engineer and technical founder. I have spent the past year on agentic systems and harness engineering, starting while I was finishing high school, where I was also wrestling team captain for two years. For most of that stretch I worked 13 to 14 hours a day on it, around school, my job and wrestling. I work across several harnesses instead of just one: OpenCode and DeepSeek Harness, Claude Desktop and Duet, Gemini Antigravity, Freebuff, ChatGPT Codex and Claude Code. I pick whichever has usage left and fits the job. Right now most of the work runs through Muse, Claude Duet and the DSH Fleet I am finishing. Three systems make up most of it. Ergon's employee agents set themselves up: the agent watches how someone works, asks them targeted questions, learns the role and keeps a memory of the person, adding skills and tools as it goes. The DSH Fleet is a distributed orchestration layer: it runs dynamic swarms across desktop, virtual machine and browser surfaces, lets outside agents join through Slack, and improves its own workflows, skills and plugin behaviour from the results of finished jobs. OpenBell is my trading research system: I set the weights of tiny models directly, freeze their predictions before scoring, and score them in time order with costs counted. 1,200+ GitHub contributions in the last year. 4,000+ automated tests across five shipped systems.

Experience

Founder — Independent AI Agent Systems

Oct 2025 – Present

- Built Ergon as a self-configuring employee-agent platform. After installation, the agent learns the employee's role by watching their recurring work and asking targeted questions, then proposes capabilities it can take over. A learned skill or tool has to pass evaluation and review before the agent can use it, so nobody has to map the role in advance or keep adjusting the agent by hand.
- Built the DSH Fleet as a distributed agent operating layer. A dynamic swarm forms around each objective, execution runs across desktop, Windows VM, phone VM and browser surfaces, and outside agents can join through Slack under the same coordination and outcome rules. Slack was not the original integration layer. I was going to build my own, but I kept getting flagged by the existing guardrails on Muse and the other platforms I build on, so I used Slack for that layer instead.
- Built OpenBell around weight-space model development. I set the weights of tiny specialist models directly, freeze their predictions before scoring, and score them in time order with costs and kill rules counted. The live trading desks and their isolated slots are where candidates get tested.
- The Omni Loop is the fleet's recursive self-improvement system, and every change it makes is gated: results from finished jobs drive revisions to the workflows, plugin, skills and the loop itself, and a revision ships only when it measures better than the current version on a fixed scorecard, checked against a frozen set of past tickets. Every change is versioned and can be rolled back.

Robert H. Lord Company (RHL) — AI Concept Discussion & Warehouse-Board Demo

Sep 2026 – Present

- Discussed a proposed Ergon deployment at RHL; the concept drew interest, but RHL has not authorised or implemented an AI integration, and no Ergon infrastructure is in production there.
- At Roy's request, built a standalone warehouse-board demo as a non-AI operational showcase. It is a demonstration artifact, not an official RHL system, AI deployment or production rollout.

Selected Projects

Ergon — Self-Configuring Employee Agents & Autonomous Role Learning

Node.js, C#, Python, SQLite

- How it onboards: once installed, the agent starts watching the employee's recurring work and tools and asking targeted questions. It builds a persistent model of the role, proposes and builds a capability, then puts it through evaluation and review before it earns the right to use it. Personalisation is built into the harness itself.
- Proven skills can enter a shared library for other agents to earn through the same gate, so capability compounds across a company while permissions stay scoped and reversible.
- Standalone product in development, with a signed Windows installer and taskbar app, demo connectors and ticket-scoped remote repair. RHL is a proposed first deployment only. Its Ergon infrastructure is still planned, not implemented. 622 automated tests pass.

DSH Fleet — Distributed, Self-Improving Agent Infrastructure

Python, TypeScript

github.com/cayleb-james2008/dsh-fleet-genesis

- The fleet is an agent control plane that spans desktop, Windows VM, phone VM and browser surfaces. Slack is the open coordination layer: agents people bring themselves can work through the same tickets and rules instead of being locked to one harness.
- Each objective gets a dynamic swarm shaped to the task, formed around a lead. The swarm works from persistent preferences, current skills, research and model routing. Every session is defined before it runs, actions that matter need approval, and the swarm dissolves after one recorded outcome.
- The fleet improves its own operating system as it works. The evidence from each job feeds Omni Loop revisions to its workflows, plugin behaviour and skill library. A revision is promoted only after a fresh check measures a win on a fixed scorecard and a frozen regression set. Failures are archived and changes stay reversible.

OpenBell — Weight-Space Training & Evaluation for Trading Models

Python, Rust

- The AI work is direct weight-space construction. I build low-dimensional weights for tiny specialist models instead of relying on prompted strategies or opaque end-to-end training. Each candidate's weights and prediction are frozen before scoring, so the hypothesis can be inspected and proven wrong.
- Evaluation is adversarial on purpose: time-ordered splits, transaction costs, negative results kept on record, kill rules set in advance, and no money test until training performance clears the breakeven for the exit shape. Conventional distillation failed on both benchmark instruments. Directly authored weights produced the strongest results, including one Nasdaq configuration that was net-positive on historical data.
- The Kalshi, futures and meme desks are live testing grounds with capital kept separate per experiment. Parallel slots and 2x ladder runs test candidates against reconciled real-market accounting,

Sophos — Coding-Agent Desktop App

React, TypeScript, Tauri

github.com/cayleb-james2008/sophos

- Desktop app that runs coding agents from saved profiles, with sessions and transcripts that survive restarts and clear errors when a provider fails.
- 1,224 interface tests, 82 Node tests and 56/56 Windows checks pass; merged after full hosted CI.

dotz — AI Coding Agent Dashboard

Rust, Tauri, JavaScript, SQLite

github.com/cayleb-james2008/dotz

- Desktop app that runs and supervises AI coding agents, with model choice and safe process-tree cleanup.

- 828+ core tests pass on each of Windows, macOS and Linux; all 6 hosted check jobs pass; merged.

Asmodeus — Trading Research on Past Data

Rust

- Tests trading ideas on real historical data in strict time order, with fees and safeguards against peeking at future data; rejected ideas are logged, not quietly dropped. 1,111 Rust tests pass. Research only.

Industry AI Suite — Source-Checked Review Tools

Python

github.com/cayleb-james2008/industry-ai-suite

- Ten local tools that review finance, research, security and operations files; every finding shows its source and a person decides. Drafts halt when sources are missing. 382 Python tests pass.

Skills

- **AI & agents:** multi-harness agent operations, autonomous role learning, continual personalisation, dynamic-swarm orchestration, recursive self-improvement systems (the Omni Loop), distributed agent infrastructure, bring-your-own-agent coordination, evaluation design, earned permissions and approval gates
- **Languages:** Python, TypeScript, JavaScript, Rust, HTML/CSS
- **Models & data:** weight-space model construction, linear/LDA specialists, chronological backtesting, experiment design, React, Tauri, SQLite, web APIs, Git/GitHub, Linux

Additional Experience

David Cooke Plaster Co. — Plaster Work | Summer 2026. Dunkin' — Crew | 2024 – 2026. Tommy's Pizza — Crew | Summer 2024.

Education

Portland High School, Portland, Connecticut — Graduated 2026

Coursework: Cybersecurity; AP Computer Science. Leadership: Wrestling Team Captain, Portland Highlanders — two years.